UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CS2023 Algoritmos y Estructuras de Datos
Árbol binario de búsqueda
Semana 6

Wilder Nina Choquehuayta

wnina@utec.edu.pe

14 de septiembre de 2026

Objetivos de la sesión



Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La estructura del nodo

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La estructura del nodo
- Buscar, Mínimo, Máximo

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La estructura del nodo
- Buscar, Mínimo, Máximo
- Insertar

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La estructura del nodo
- Buscar, Mínimo, Máximo
- Insertar
- Sucesor y Predecesor

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La estructura del nodo
- Buscar, Mínimo, Máximo
- Insertar
- Sucesor y Predecesor
- Eliminar — el más delicado de todos

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La estructura del nodo
- Buscar, Mínimo, Máximo
- Insertar
- Sucesor y Predecesor
- Eliminar — el más delicado de todos
- Recorrido-Inorder

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La clase pasada quedó en teoría

Vimos el TDA completo y su complejidad $O(h)$ — pero nunca vimos **cómo** se implementa cada operación realmente.

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La clase pasada quedó en teoría

Vimos el TDA completo y su complejidad $O(h)$ — pero nunca vimos **cómo** se implementa cada operación realmente.

¡Piénsalo!

De las 7 operaciones del TDA, ¿cuál crees que va a ser la más simple de implementar? ¿Y la más delicada?

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La clase pasada quedó en teoría

Vimos el TDA completo y su complejidad $O(h)$ — pero nunca vimos **cómo** se implementa cada operación realmente.

¡Piénsalo!

De las 7 operaciones del TDA, ¿cuál crees que va a ser la más simple de implementar? ¿Y la más delicada?

Spoiler

BUSCAR es casi trivial. ELIMINAR — por los casos que hay que manejar — es, de lejos, la más delicada. Hoy construimos las 7, en ese orden de dificultad creciente.

La estructura

El nodo

Campos

Cada nodo x guarda:

El nodo

Campos

Cada nodo x guarda:

- $x.clave$: el valor almacenado.

El nodo

Campos

Cada nodo x guarda:

- $x.clave$: el valor almacenado.
- $x.izq$, $x.der$: punteros a los hijos (nulos si no existen).

El nodo

Campos

Cada nodo x guarda:

- $x.clave$: el valor almacenado.
- $x.izq$, $x.der$: punteros a los hijos (nulos si no existen).
- $x.padre$: puntero al padre (nulo si x es la raíz).

El nodo

Campos

Cada nodo x guarda:

- $x.clave$: el valor almacenado.
- $x.izq$, $x.der$: punteros a los hijos (nulos si no existen).
- $x.padre$: puntero al padre (nulo si x es la raíz).

¡Piénsalo!

En la lista doblemente enlazada, guardar el puntero al anterior nos ahorraba $O(n)$ al eliminar. ¿Qué nos va a ahorrar $x.padre$ aquí?

El nodo

Campos

Cada nodo x guarda:

- $x.clave$: el valor almacenado.
- $x.izq$, $x.der$: punteros a los hijos (nulos si no existen).
- $x.padre$: puntero al padre (nulo si x es la raíz).

¡Piénsalo!

En la lista doblemente enlazada, guardar el puntero al anterior nos ahorraba $O(n)$ al eliminar. ¿Qué nos va a ahorrar $x.padre$ aquí?

Respuesta

Subir desde cualquier nodo sin tener que volver a bajar desde la raíz — esencial para SUCESOR y ELIMINAR.

Búsqueda y extremos

Buscar

Algoritmo 1: BUSCAR(x, k)

```
si x = NULO o k = x.clave entonces
    devolver x
si k < x.clave entonces
    devolver BUSCAR(x.izq, k)
devolver BUSCAR(x.der, k)
```

Buscar

Algoritmo 2: BUSCAR(x, k)

si $x =$ NULO **o** $k = x.clave$ **entonces**
  devolver x
si $k < x.clave$ **entonces**
  devolver BUSCAR$(x.izq, k)$
devolver BUSCAR$(x.der, k)$

Exactamente la traducción directa de la propiedad BST: compara, y sigue por el lado correcto. $O(h)$: baja, a lo más, un camino completo.

Mínimo y Máximo

Algoritmo 3: MÍNIMO(x)

mientras $x.izq \neq$ NULO **hacer**
  $x \leftarrow x.izq$

devolver x

Mínimo y Máximo

Algoritmo 4: MÍNIMO(x)

mientras $x.izq \neq$ NULO **hacer**
  $x \leftarrow x.izq$

devolver x

¡Piénsalo!

¿Por qué basta con ir siempre a la izquierda, sin comparar ningún valor?

Mínimo y Máximo

Algoritmo 5: MÍNIMO(x)

mientras $x.izq \neq$ NULO **hacer**
  $x \leftarrow x.izq$

devolver x

¡Piénsalo!

¿Por qué basta con ir siempre a la izquierda, sin comparar ningún valor?

Respuesta

Por la propiedad BST, todo lo que está a la izquierda de un nodo es **menor** — el nodo más a la izquierda no tiene nada menor que él. MÁXIMO es simétrico, yendo siempre a la derecha.

Insertar

Insertar

Algoritmo 6: INSERTAR(T, k)

```
y ← NULO, x ← T.raiz
mientras x ≠ NULO hacer
    y ← x
    si k < x.clave entonces
        x ← x.izq
    en otro caso
        x ← x.der
z ← nuevo nodo con clave k, z.padre ← y
si y = NULO entonces
    T.raiz ← z
si no, si k < y.clave entonces
    y.izq ← z
en otro caso
    y.der ← z
```

Insertar: la idea

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¡Piénsalo!

El ciclo `while` se parece mucho a BUSCAR. ¿Qué está buscando en realidad, si k **no** está todavía en el árbol?

Insertar: la idea

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

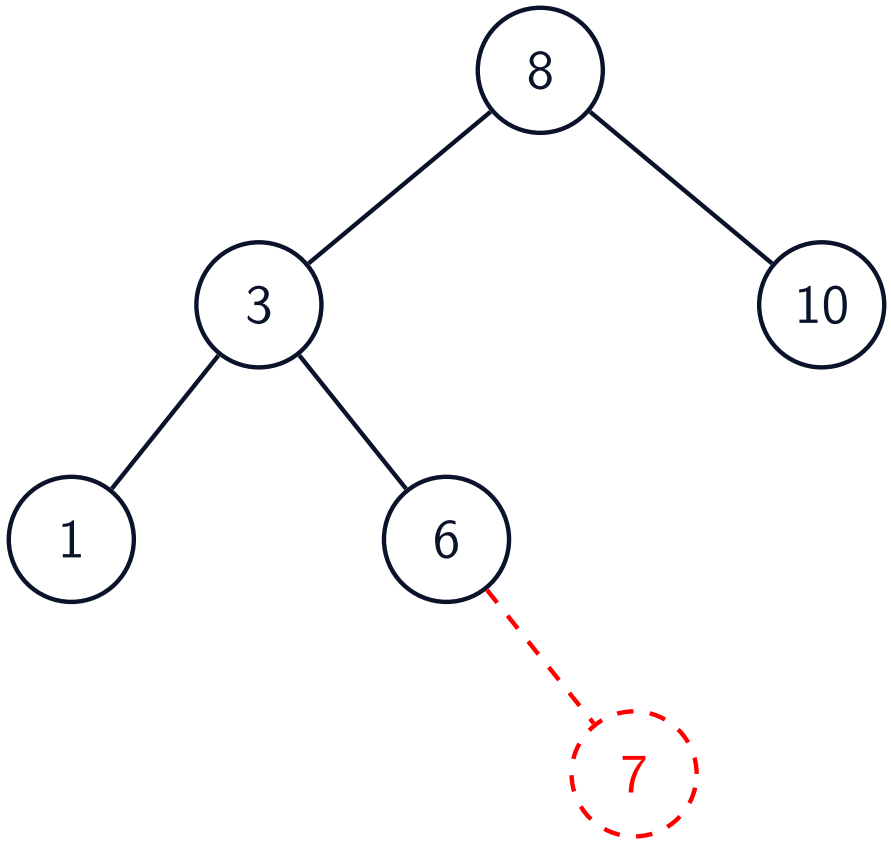
¡Piénsalo!

El ciclo `while` se parece mucho a BUSCAR. ¿Qué está buscando en realidad, si k **no** está todavía en el árbol?

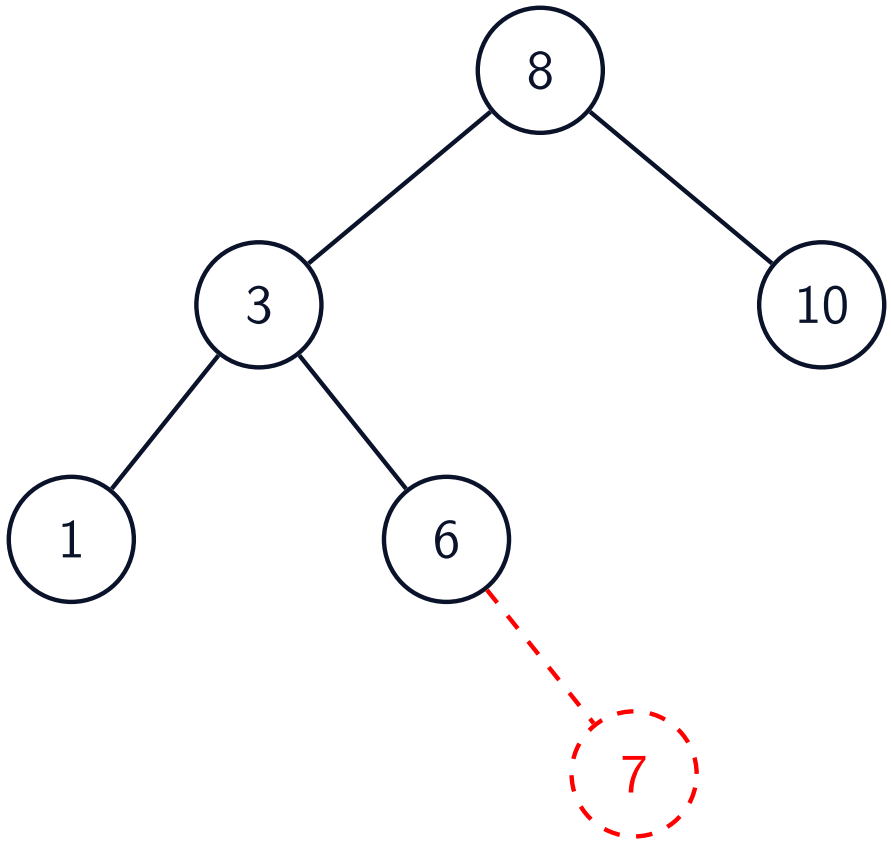
Respuesta

Busca el lugar donde **debería** estar: el ciclo termina justo cuando cae en un puntero nulo — ahí es donde cuelga el nodo nuevo, como hijo de y (el último nodo visitado).

Insertando, paso a paso



Insertando, paso a paso



Insertando 7: 8 $\rightarrow$ izquierda ($7 < 8$); 3 $\rightarrow$ derecha ($7 > 3$); 6 $\rightarrow$ derecha ($7 > 6$); nulo $\rightarrow$ aquí cuelga el nuevo nodo, como hijo derecho de 6.

Sucesor y Predecesor

Sucesor

Algoritmo 7: SUCESOR(x)

si $x.der \neq$ NULO **entonces**
    devolver MÍNIMO($x.der$)
$y \leftarrow x.padre$
mientras $y \neq$ NULO **y** $x = y.der$ **hacer**
    $x \leftarrow y$
    $y \leftarrow y.padre$
devolver y

Sucesor: dos casos

¡Piénsalo!

Si x **tiene** hijo derecho, ¿dónde está su sucesor? Y si **no** tiene hijo derecho, ¿cómo lo encuentras sin volver a bajar desde la raíz?

Sucesor: dos casos

¡Piénsalo!

Si x **tiene** hijo derecho, ¿dónde está su sucesor? Y si **no** tiene hijo derecho, ¿cómo lo encuentras sin volver a bajar desde la raíz?

Caso 1: hay hijo derecho

El sucesor es el **mínimo** de ese subárbol derecho — el siguiente valor mayor siempre está ahí.

Sucesor: dos casos

¡Piénsalo!

Si x **tiene** hijo derecho, ¿dónde está su sucesor? Y si **no** tiene hijo derecho, ¿cómo lo encuentras sin volver a bajar desde la raíz?

Caso 1: hay hijo derecho

El sucesor es el **mínimo** de ese subárbol derecho — el siguiente valor mayor siempre está ahí.

Caso 2: no hay hijo derecho

Sube por los punteros *padre* mientras vengas de un hijo **derecho**; el primer ancestro del que vengas por la **izquierda** (o la raíz misma) es el sucesor.

Sucesor: dos casos

¡Piénsalo!

Si x **tiene** hijo derecho, ¿dónde está su sucesor? Y si **no** tiene hijo derecho, ¿cómo lo encuentras sin volver a bajar desde la raíz?

Caso 1: hay hijo derecho

El sucesor es el **mínimo** de ese subárbol derecho — el siguiente valor mayor siempre está ahí.

Caso 2: no hay hijo derecho

Sube por los punteros *padre* mientras vengas de un hijo **derecho**; el primer ancestro del que vengas por la **izquierda** (o la raíz misma) es el sucesor.

PREDECESOR es exactamente simétrico: izquierda en vez de derecha, en todos los pasos.

Eliminar

Eliminar: tres casos

¡Piénsalo!

Antes de ver código: si el nodo z que quieres eliminar tiene 0, 1, o 2 hijos, ¿en qué se complica cada caso?

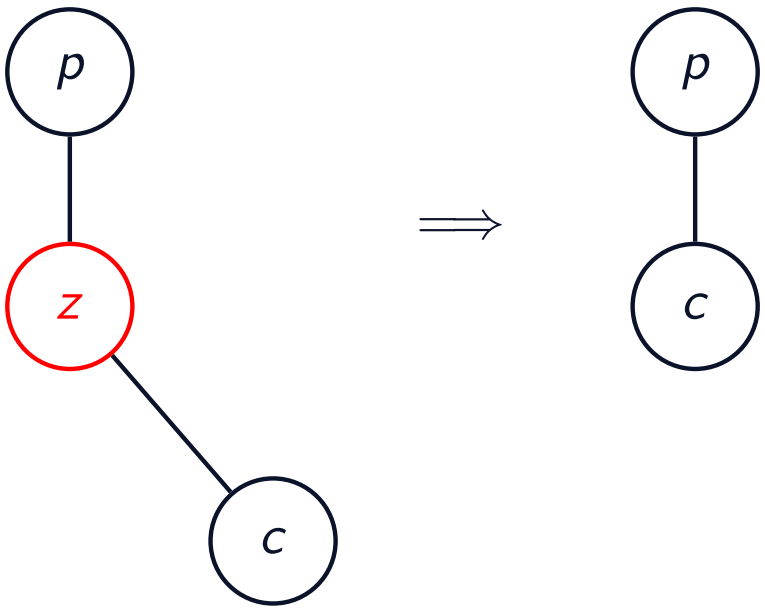
Eliminar: tres casos

¡Piénsalo!

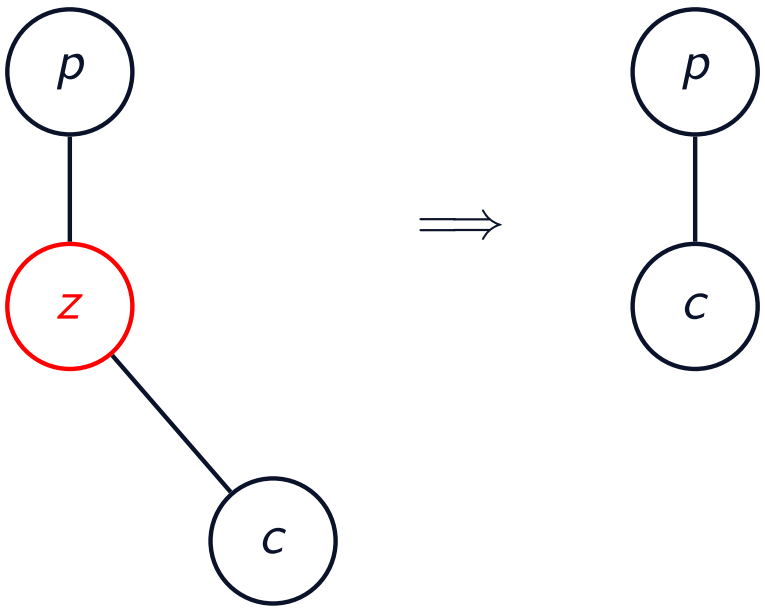
Antes de ver código: si el nodo z que quieres eliminar tiene 0, 1, o 2 hijos, ¿en qué se complica cada caso?

Con 0 o 1 hijo, es fácil: el padre de z simplemente "adopta" a ese único hijo (o a nada). El problema real es con 2 hijos: no puedes simplemente conectar los dos directamente al padre de z.

Caso 1 y 2: cero o un hijo



Caso 1 y 2: cero o un hijo



El padre de z pasa a apuntar directamente a c (el único hijo de z, o nulo si no tenía ninguno) — z simplemente se "salta".

El helper: Transplantar

Algoritmo 8: TRANSPLANTAR(T, u, v)

```
// Reemplaza el subárbol de u por el de v
si u.padre = NULO entonces
    T.raiz ← v
si no, si u = u.padre.izq entonces
    u.padre.izq ← v
en otro caso
    u.padre.der ← v
si v ≠ NULO entonces
    v.padre ← u.padre
```

El helper: Transplantar

Algoritmo 9: TRANSPLANTAR(T, u, v)

```
// Reemplaza el subárbol de u por el de v
si u.padre = NULO entonces
    T.raiz ← v
si no, si u = u.padre.izq entonces
    u.padre.izq ← v
en otro caso
    u.padre.der ← v
si v ≠ NULO entonces
    v.padre ← u.padre
```

No mueve datos: solo reconecta punteros — exactamente lo que necesitábamos en el caso de 0 o 1 hijo.

Caso 3: dos hijos

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¡Piénsalo!

No puedes simplemente conectar ambos hijos de z al padre de z (¿a dónde iría el segundo?). ¿Qué **otro** valor del árbol podría ocupar el lugar de z sin romper la propiedad BST?

Caso 3: dos hijos

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¡Piénsalo!

No puedes simplemente conectar ambos hijos de z al padre de z (¿a dónde iría el segundo?). ¿Qué **otro** valor del árbol podría ocupar el lugar de z sin romper la propiedad BST?

Respuesta: el sucesor de z

El $\textsc{Sucesor}(z)$ — el mínimo del subárbol derecho — es el valor **más cercano** a z por arriba. Ponerlo en el lugar de z mantiene el orden: nada a la izquierda es mayor, nada a la derecha es menor.

Caso 3: dos hijos

¡Piénsalo!

No puedes simplemente conectar ambos hijos de z al padre de z (¿a dónde iría el segundo?). ¿Qué **otro** valor del árbol podría ocupar el lugar de z sin romper la propiedad BST?

Respuesta: el sucesor de z

El $\textsc{Sucesor}(z)$ — el mínimo del subárbol derecho — es el valor **más cercano** a z por arriba. Ponerlo en el lugar de z mantiene el orden: nada a la izquierda es mayor, nada a la derecha es menor.

Y como es un **mínimo**, el sucesor nunca tiene hijo izquierdo — eliminarlo de su posición original cae en el Caso 1 o 2, que ya resolvimos.

Eliminar

Algoritmo 10: ELIMINAR(T, z)

```
si z.izq = NULO entonces
    TRANSPLANTAR(T, z, z.der)
si no, si z.der = NULO entonces
    TRANSPLANTAR(T, z, z.izq)
en otro caso
    y ← MÍNIMO(z.der)
    si y.padre ≠ z entonces
        TRANSPLANTAR(T, y, y.der)
        y.der ← z.der,  y.der.padre ← y
    TRANSPLANTAR(T, z, y)
    y.izq ← z.izq,  y.izq.padre ← y
```

Eliminar: leyendo el caso 3 con cuidado

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¡Piénsalo!

¿Por qué el código distingue si $y.padre = z$ (el sucesor es **hijo directo** de z) del caso general?

Eliminar: leyendo el caso 3 con cuidado

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¡Piénsalo!

¿Por qué el código distingue si $y.padre = z$ (el sucesor es **hijo directo** de z) del caso general?

Respuesta

Si y ya es el hijo derecho directo de z, no hace falta "sacarlo" de ningún lado — simplemente se convierte en la nueva raíz del subárbol. Si y está **más abajo**, primero hay que desconectarlo de su posición original ($\textsc{Transplantar}(T, y, y.der)$) antes de moverlo.

Recorrido inorder

Recorrido-Inorder

Algoritmo 11: RECORRIDO-INORDER(x)

```
si x ≠ NULO entonces
    RECORRIDO-INORDER(x.izq)
    visitar x
    RECORRIDO-INORDER(x.der)
```

Recorrido-Inorder

Algoritmo 12: RECORRIDO-INORDER(x)

si $x \neq$ NULO **entonces**
    RECORRIDO-INORDER($x.izq$)
    visitar x
    RECORRIDO-INORDER($x.der$)

Tres líneas, y por la propiedad BST (que vimos la clase pasada) esto **siempre** visita los elementos en orden creciente. $\Theta(n)$: visita cada nodo exactamente una vez.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El nodo guarda *clave*, *izq*, *der* y *padre* — este último es lo que permite SUCESOR y ELIMINAR sin volver a bajar desde la raíz.
- BUSCAR, MÍNIMO/MÁXIMO e INSERTAR son variaciones directas de "sigue la propiedad BST hasta un puntero nulo".
- SUCESOR tiene dos casos: mínimo del subárbol derecho si existe, o subir hasta el primer ancestro del que vengas por la izquierda.
- ELIMINAR tiene tres casos (0, 1 o 2 hijos); el caso de 2 hijos se resuelve reemplazando z por su SUCESOR — que, al ser un mínimo, siempre cae en el Caso 1 o 2 al eliminarlo de su lugar original.
- RECORRIDO-INORDER es recursivo y directo: izquierda, visitar, derecha.

Logros de la sesión

Logros de la sesión

- Sabemos implementar las 7 operaciones del TDA BST con pseudocódigo real.

Logros de la sesión

- Sabemos implementar las 7 operaciones del TDA BST con pseudocódigo real.
- Sabemos por qué guardar *x.padre* simplifica SUCESOR y ELIMINAR.

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos implementar las 7 operaciones del TDA BST con pseudocódigo real.
- Sabemos por qué guardar *x.padre* simplifica SUCESOR y ELIMINAR.
- Sabemos manejar los 3 casos de ELIMINAR, incluyendo por qué el caso de 2 hijos se reduce a los otros dos vía el sucesor.

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias